CS 499 Milestone Two: Software Engineering and Design Enhancement

Matthew Wood

Southern New Hampshire University

Artifact Description

The artifact I selected for my software engineering and design enhancement is the Travlr Getaways full-stack web application that I originally developed in CS 465: Full Stack Development I. The project was created during my computer science program as a MEAN stack application using MongoDB, Express, Angular, and Node.js. The customer-facing portion of the application displays travel packages, while the administrative side allows an authenticated user to add, edit, and delete trip information through a REST API. The original project already demonstrated full-stack development, API communication, database access, and JWT-based authentication.

Justification for Inclusion and Enhancement

I selected Travlr Getaways because it is one of my more complete projects and gives me an opportunity to show how I can improve an existing application instead of only building something that works once. For this milestone, I focused on software engineering practices that make the application easier to maintain, more reliable, and safer to use. I replaced hard-coded API configuration with environment-based configuration, changed mixed Promise and async code to a more consistent async/await approach, and corrected the single-trip API request so it returns one trip object instead of an array. I also centralized the Angular API configuration and authentication headers so the same logic does not have to be repeated throughout the service.

Another major part of the enhancement was validation and error handling. I added stronger validation to the Angular add and edit forms for fields such as trip code, price, names, and description length. I also added server-side validation so invalid data cannot bypass the browser and be sent directly to the API. API errors now return controlled messages instead of exposing raw application errors. I also improved JWT authentication error handling so an expired or invalid token returns an unauthorized response without exposing an internal stack trace or local file paths. These changes show improvement in code organization, defensive programming, input validation, configuration management, and security.

Course Outcomes

The enhancement most directly supports Course Outcome Four because I used current development techniques and tools to improve an existing computing solution. Instead of changing the application only for appearance, I refactored the way the client and server communicate, reduced duplicated logic, improved validation, and made error handling more consistent. The application still performs its original functions, but the implementation is easier to understand and maintain.

The enhancement also strongly supports Course Outcome Five by applying a security mindset to the application. Client-side validation improves the user experience, but I did not rely on it as the only protection. I added equivalent server-side checks because API requests can be sent without using the Angular interface. I also kept JWT protection on the create, update, and delete routes and changed authentication and API error handling so internal application details are not returned to the client. This milestone provides some evidence for the communication-related outcomes through the code review, documentation, and ePortfolio explanation, but I am

not treating the algorithms and data structures outcome as a primary part of this enhancement. That outcome will be addressed more directly in the next enhancement.

Reflection

One of the main things I learned from this enhancement is that improving an existing project requires a different mindset than getting the original assignment to work. In the original course, the main goal was to build the required full-stack functionality. During the capstone review, I was able to look at the same code as a larger system and notice issues such as hard-coded configuration, duplicated authentication logic, inconsistent asynchronous code, weak validation, and error responses that revealed too much information.

The most useful challenge was making changes without breaking the functionality that already worked. Some changes affected multiple parts of the application. For example, changing the API method for a single trip from returning an array to returning one object also required updating the Angular edit component to work with the new response. I tested the application after the changes by retrieving trips, retrieving a single trip, checking a missing trip, loading the travel page, and completing add, edit, and delete operations. I also tested invalid form input, invalid API input, valid authentication, and expired authentication tokens. This process reinforced the importance of regression testing whenever code is refactored.

Overall, this enhancement improved the Travlr Getaways project without changing its purpose. The work demonstrates that I can review an existing application, identify maintainability and security weaknesses, make targeted improvements, and verify that the original behavior still works. I intentionally left database-specific schema and indexing changes and the more advanced search, filtering, and sorting logic for later milestones so each enhancement can demonstrate a distinct area of computer science.